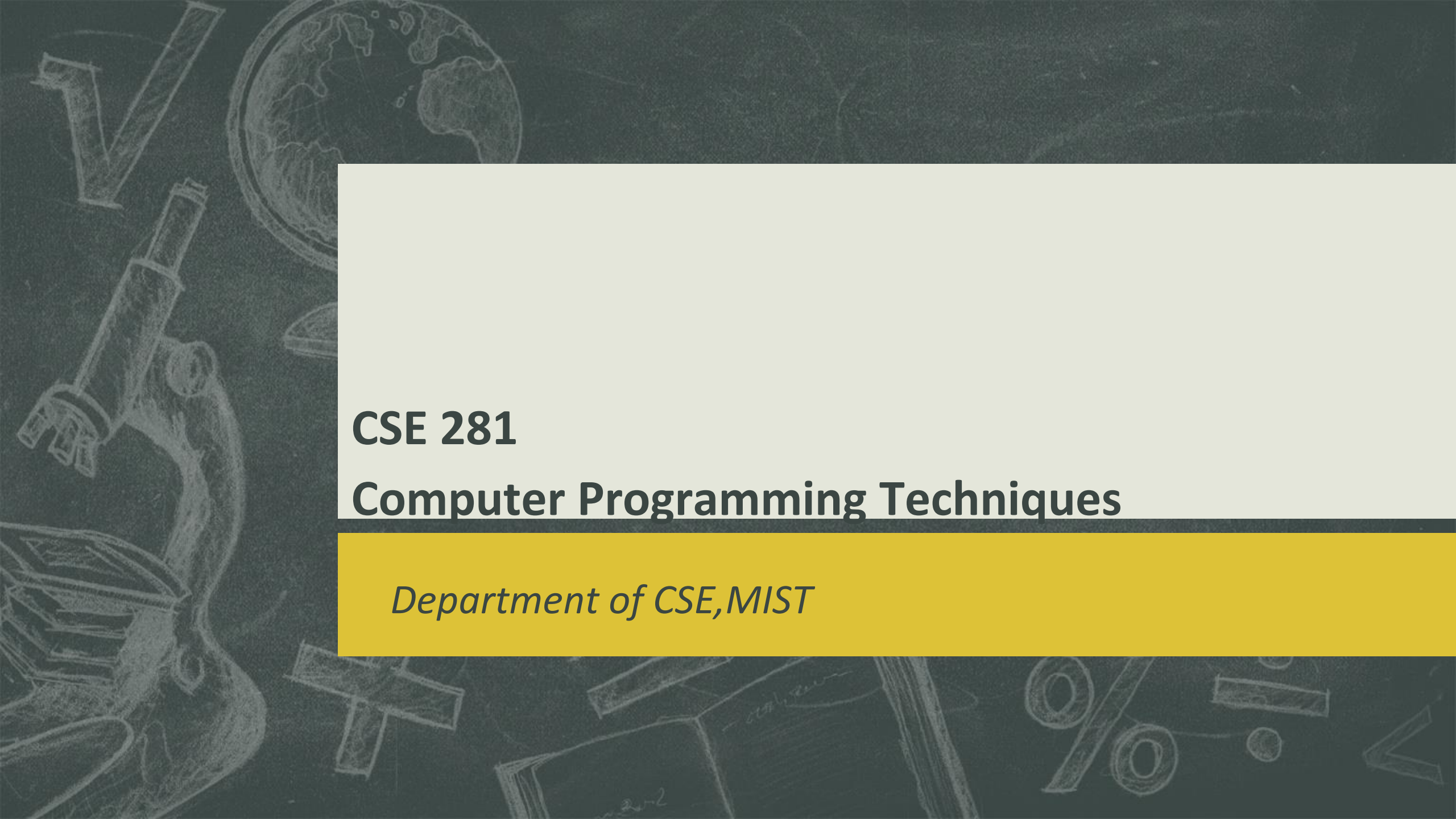


The background is a dark, textured surface resembling a chalkboard. It features faint, light-colored chalk drawings of various scientific and technical concepts. On the left, there is a large, detailed drawing of a microscope. Above it, a globe of the Earth is visible. To the right of the globe, there are some abstract, curved lines. In the bottom right corner, there are drawings of mathematical symbols, including a percentage sign (%), an exclamation mark (!), and a less-than sign (<).

CSE 281

Computer Programming Techniques

Department of CSE, MIST

Why are Arrays Necessary?

- Suppose you want to store the grades/marks of your underGrad courses.[Let you have to take 60 courses in your entire undergrad study]
 - How many variables do you have to declare?
- Again, Suppose, I want to store the Ct 1 marks of 60 students of your class.
 - How many variables do I have to declare?
- In either case, we can **solve** it by creating **60 variable individually** but, giving unique names to 60 variables and operating with them is tedious and impractical.
- This type of scenarios, where a number of similar types of variables are required can be solved by using **Array** in C programming.

Array

- A list of same type variables which can be accessed through a **common name**
- An **individual variable** in the array is called an ***array element***.
- We can divide Arrays in 2 types:
 - One dimensional Array
 - Multi-dimensional Array[More than two dimensional array generally not used]

Array

- Array declaration requires : array type, name, size(size specifies the number of array elements)

- **Array Declaration:**

Data_type array_name [array_size];

- **Array Declaration Example:**

float

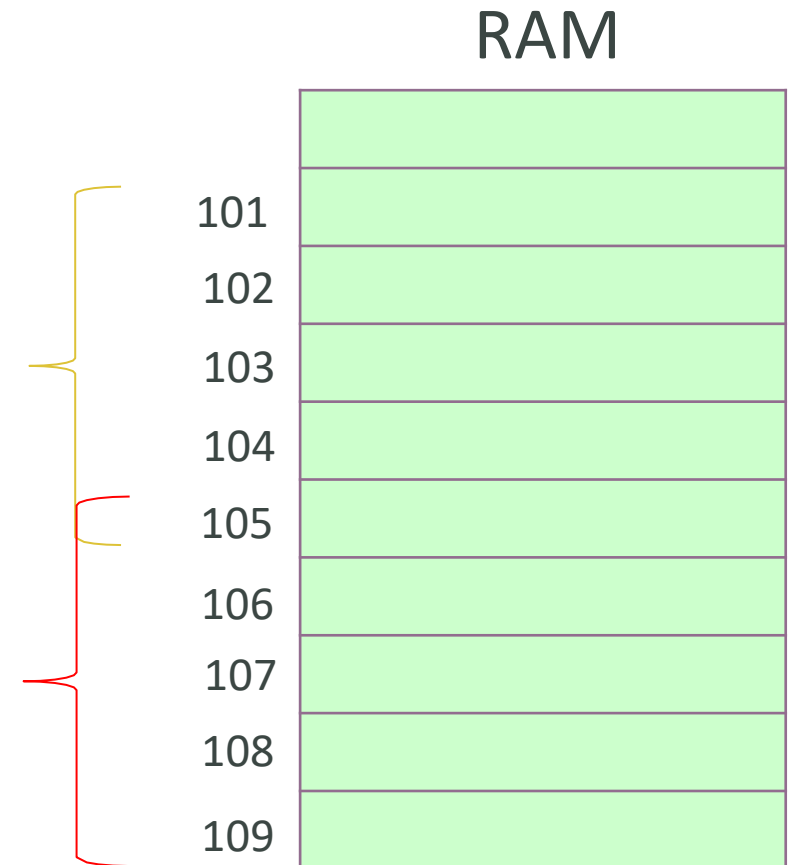
marks[60]; int

a[20];

Char name[30];

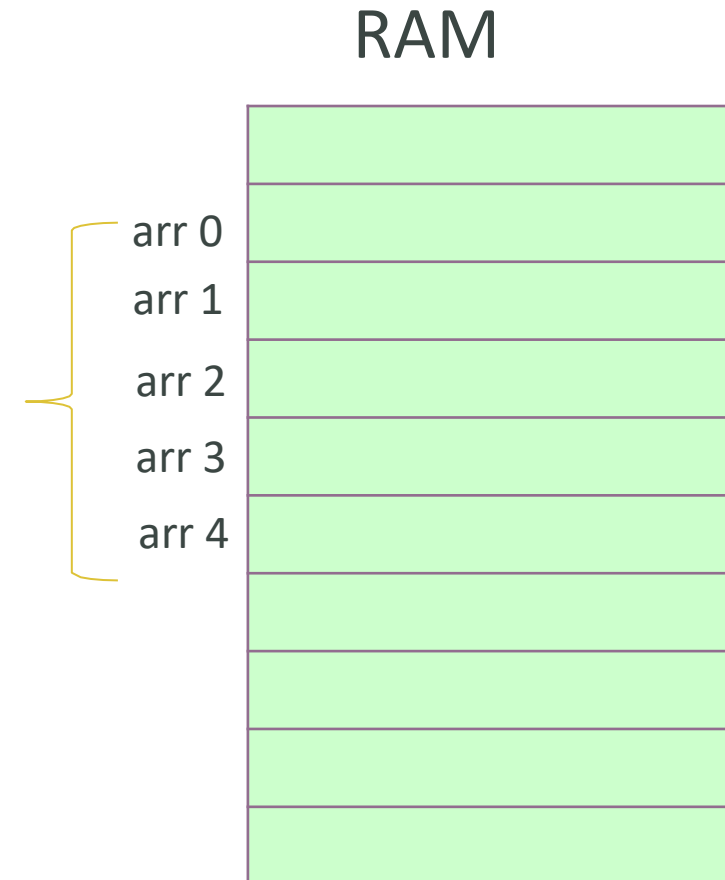
Array Declaration Detail

- What happens when we declare an array of 5 integer elements such as `int arr[5]`?
- Ans – **Consecutive memory** for 5 integer element in the RAM is provided where first element is assigned at the lowest address



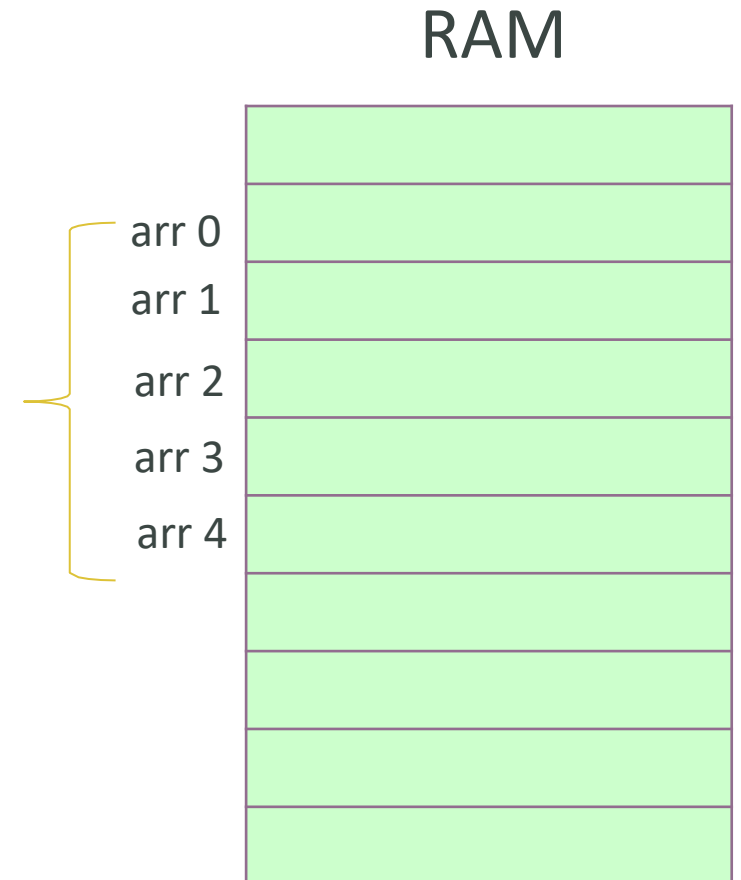
Array Declaration Detail

- when we declare an array of 5 integer elements such as `int arr[5]`, we have 5 integer elements
- We can access each of the variables with the **same array name** and **their position**
- **first element** of the array is in index 0, accessed by `arr[0]`
- **Second element** is in index 1, accessed by `arr[1]`
- **Third element** is in index 2, accessed by `arr[2]`
- **Fourth element** is in index 3, accessed by `arr[3]`
- **Fifth element** is in index 4, accessed by `arr[4]`
- Remember array indexing starts at 0 in C programming



Array Declaration Detail

- If you declare an array of n elements, you can access k^{th} element where $k \leq n$, by the index $[k-1]$
- If you have declared an array of 5 elements, you can access element 1-5 by `arr[0]` to `arr[4]` respectively. But if you want to access the index **`arr[5]`** or **`arr[10]`**, you may get compilation error or garbage values



Initializing Array

Process 1 : `int arr[5] = {5,10,11,15,20};`

Process 2 : `int arr[ ] = {5,10,11,15,20};`

Process 3 : `int arr[5] = {5,10};`

RAM	
0	5
1	10
2	11
3	15
4	20

Initializing Array

Process 3 : `int arr[5] = {5,10};`

**Notice that it auto initialized the other indexes to 0.
However, just declaring an array like “arr[5]” wouldn’t have
done that and the indexes would have garbage values.**

0
1
2
3
4

RAM

5
10
0
0
0

Array Example 2

- **Take input from user and print the array elements**

```
#include<stdio.h> int main(){
    int arr[3];
    scanf("%d",&arr[0]);
    scanf("%d",&arr[1]);
    scanf("%d",&arr[2]);
    printf("%d", arr[0]);
    printf("%d", arr[1]);
    printf("%d", arr[2]);
}
```

Array Example 2 Alternate Solution

- **Take input from user and print the array elements**

```
#include<stdio.h> int main()
{
    int arr[3];
    scanf("%d%d%d",&arr[0],&arr[1],&arr[2]);
    printf("%d", arr[0]);
    printf("%d", arr[1]);
    printf("%d", arr[2]);
}
```

Array Example 2 Alternate Solution Using Loop

- **Take input from user and print the array elements**

```
#include<stdio.h> int main(){
    int arr[3], i;
    for(i=0; i<3 ; i++){
        scanf("%d",&arr[i]);
    }
    for(i=0; i<3 ; i++){
        printf("%d", arr[i]);
    }
}
```

Array Example 3

- Specify the number 'n' of inputs you want to take. Take input of n elements and print them

```
int main(){
    int n; scanf("%d",&n);
    int arr [n];
    /*you may declare an array of the
    size of a variable*/
    for(int i=0; i<n ; i++){
        scanf("%d",&arr[i]);
    }
    for(int i=0; i<n ; i++){
        printf("%d", arr[i]);
    }
}
```

Array Example 4

- **Take input of n integer numbers, Calculate and print the sum of them**

```
int main(){
    int arr[100],n,sum=0;
    scanf("%d",&n);
    for(int i=0; i<n ; i++){
        scanf("%d",&arr[i]);
    }
    for(int i=0; i<n ; i++){
        sum = sum+arr[i];
    }
    printf("Summation of n elements of the array is : %d", sum);
}
```

Array Example 5

- **Take input of n integer numbers, Find and print the minimum number**

```
int main(){
    int arr[100],n,min;
    scanf("%d",&n); for(int
    i=0; i<n ; i++){
        scanf("%d",&arr[i]);
    }
    min = arr[0];
    for(int i=0; i<n ; i++){
        if(min > arr[i])
            min = arr[i];
    } printf("Min number is: %d", min);
}
```

Array Example 6

- **Take input of n integer numbers, Find and print the maximum number**

```
int main(){
    int arr[100],n,max;
    scanf("%d",&n); for(int
    i=0; i<n ; i++){
        scanf("%d",&arr[i]);
    }
    max = arr[0];
    for(int i=0; i<n ; i++){
        if(max < arr[i])
            max = arr[i];
    }
}
```

Array Example 7

- **Copy the elements of one array to another array**

```
int main(){
    int arr[100],arr2[100],n,max;
    scanf("%d",&n); for(int i=0; i<n ; i++){
        scanf("%d",&arr2[i]);
    }
    for(int i=0; i<n ; i++){
        arr[i] = arr2[i];
    }
}
```

Array Example 8

A program to take input of CT1-3 and Midterm marks for n students and then finding out the mark to be counted (Best 2 CTs + Mid).

Tasks

- Declare an array of 5 float numbers and print them
- Declare an array of float numbers, take user input of n numbers and print them
- Declare an array of float numbers, take user input of n numbers, calculate and print the average of them
- Store the square of the integer numbers 1-10 in an array and print the array
- Take two arrays of integer of size 5 and find out the UNION set of the 2 arrays

Character Array

- The most common use of the one-dimensional array in C programming is character array
- Declaration of character array:
`char array_name[array_size];`

Initializing Character Array

- Suppose you want to store the word 'hello' in a character array. How will you do that?
- Step 1: declare character array of size 5(five characters)
- Store the first character 'h' in array index 0
- Store the other characters in the same way

```
char arr[5];  
arr[0] = 'h';  
arr[1] = 'e';  
arr[2] = 'l';  
arr[3] = 'l';  
arr[4] = 'o';
```

Character Array Printing

- **store the word 'hello' in a character array and print it**

```
int main(){
    char arr[5];
    arr[0] = 'h';
    arr[1] = 'e';
    arr[2] = 'l';
    arr[3] = 'l';
    arr[4] = 'o';

    for(int i=0; i<5;i++){
        printf("%c",arr[i]);
    }
}
```

String

- String is defined as a ***NULL terminated*** character array.
- One **basic difference** between character array and string is, **string MUST be terminated by a null character.**

Character
Array

h	e	l	l	o		
---	---	---	---	---	--	--

String

h	e	l	l	o	\0	
---	---	---	---	---	----	--

String

- Suppose you want to store the word 'hello' in a string. How will you do that?
- Step 1: declare character array of size 6(five characters+null)
- Store the first character 'h' in array index 0
- Store the other characters in the same way
- Store the “null character” at the end

```
char arr[6];  
arr[0] = 'h';  
arr[1] = 'e';  
arr[2] = 'l';  
arr[3] = 'l';  
arr[4] = 'o';  
arr[5] = '\0';
```

String Input-Output

- Format specifier for string input output is : “%s”
- **Take input of a string and print it**

```
int main(){  
    char str[10];  
    scanf("%s",str);  
    printf("%s",str)  
};
```

Note: The & is not required in string input

Input:
hello

h	e	l	l	o	\0	
---	---	---	---	---	----	--

Output:
hello

Note: When you take input of a string, you do not have to put the null character explicitly. The compiler does it itself.

String Input-Output Cntd.

- Suppose you want to take input of a sentence like “**hello world**” using scanf statement, what you would do?
- Declare a string of size 12
- Take input using scanf
- Print the output

h	e	l	l	o		w	o	r	l	d	\0	
---	---	---	---	---	--	---	---	---	---	---	----	--

```
int main(){  
  
char str[12];  
scanf("%s",str)  
;  
printf("%s",str)  
;  
}
```

What will be the output of this code? Let's run it in codeblocks....

String Input-Output Cntd.

- Suppose you want to take input of a sentence like “**hello world**” using scanf statement, what you would do?
- Declare a string of size 12
- Take input using scanf
- Print the output

h	e	l	l	o		w	o	r	l	d	\0	
---	---	---	---	---	--	---	---	---	---	---	----	--

```
int main(){  
  
char str[12];  
scanf("%s",str)  
;  
printf("%s",str)  
;  
}
```

What did we do wrong in the code?

```
hello world  
hello  
  
Process returned 0 (0x0)  
Press any key to continue.
```

String Input-Output Cntd.

- Scanf terminates if it encounters any **white space** during taking input
- **White space examples:** space, tabs, carriage return, newline etc
- In the previous code, when we entered a space(" ") after the word "hello" scanf statement terminated, means it stopped taking any further input
- That's why our output printed upto the word "hello"
- How to solve this problem?
- C programming provides i/o functions to allow taking input and printing output with whitespace
 - gets() for taking input with whitespaces
 - puts() for printing output with whitespaces

String Input-Output Cntd.

- Try solving this code with `gets()` and `puts()` function now

```
int main(){  
  
char str[12];  
scanf("%s",str);  
printf("%s",str);  
}
```

h	e	l	l	o		w	o	r	l	d	\0	
---	---	---	---	---	--	---	---	---	---	---	----	--

```
int main(){  
  
char str[12];  
gets(str);  
puts(str);  
}
```

Output is correct
now

```
hello world  
hello world  
  
Process returned 0 (0x0)  
Press any key to continue.
```

Accessing Elements in a String

- **Let this be your string:** `int main(){`

0	1	2	3	4	5	6	7	8	9	10	11
h	e	l	l	o		w	o	r	l	d	\0

- The first character 'h' is in Index 0
- The last character 'd' is in index 10
- You can access any element at any index just like a character array

```
char str[12];
gets(str);
for(int i=0 ; i<11 ; i++){
    printf("%c",str[i];
}
}
```

String Example 1

- Take input of a string and calculate its length

```
int main(){ char  
str[100]; int len=0;
```

```
gets(str);
```

```
for(int i=0 ; str[i]!='\0';i++)
```

```
{
```

```
    len++;
```

```
}
```

```
printf("length of the string \"%s\" is %d",str,len);
```

```
}
```

Notice how the NULL terminator (\0) helps us to traverse the str using a for loop!

String Example 2

- **Take input of a sentence and calculate the number of words in it**[Assume there is no space before the first word, every other word is separated by a single space]

```
▪ int main(){
char str[100];
int word=0;
gets(str);
for(int i=0 ; str[i]!='\0' ; i++){
    if(str[i]==' '){
        word++;    }
}
word++;
printf("Number of words in the sentence \"%s\" is %d",str,word);
}
```

String Example 3

- **Take two strings as input and copy the elements of string 2 in string 1.**

```
int main(){
char str1[50],str2[50];
gets(str1);
gets(str2);

for(int i=0 ; str2[i]!='\0' ; i++){
    str1[i] = str2[i];
}
printf("string 1 is: \"%s\" \n",str1);
printf("string 2 is: \"%s\" ",str2);

}
```

String Example 4

- **Take two strings as input and print out the concatenation of them.**

```
int main(){  
char str1[50],str2[50];  
int len = 0;
```

```
gets(str1);  
gets(str2);
```

```
for(int i=0;str1[i]!='\0';i++){  
    len++;  
}
```

```
for(int i=0 ; str2[i]!='\0' ; i++){  
    str1[len++] = str2[i];  
}  
str1[len]='\0';  
printf("Concatenation is: \"%s\" \n",str1);  
}
```

String-Handling Functions

C/C++ have multiple built in function to make your life easier.

- ❑ `strlen ( string_name )` //simply returns string's length
- ❑ `strcpy( string1 , string2 )` //copies string2 to string 1
- ❑ `strcat( string1, string2 )` // adds string 2 after string 1
- ❑ `strrev( string_name )` //reverses the string

and more....

String-Handling Functions

strcmp(string1, string2) //compares both string and returns:

Return Value	Remarks
0	if both strings are identical (equal)
negative	if the ASCII value of first unmatched character is less than second.
positive integer	if the ASCII value of first unmatched character is greater than second.

The function returns the exact value -1 for negative and +1 for positive

String-Handling Functions

More String Functions

Task

- Take input of a string, take input of a character and check the frequency of the character in that string
- Write a program in C to convert a string to lowercase.
- Write a program in C to convert a string to uppercase.
- Take input of two strings and compare if they are same string or not
- Take a string as input and print it in reverse order.
- Take a string as input and check whether it is palindrome or not.
- Take input of a string and count the number of unique characters

Task

- Take input of a string, take input of a character and check the frequency of the character in that string

```
int main(){
char str[50],ch;
int freq=0;
gets(str);
scanf("%c",&ch);
for(int i=0 ;str[i]!='\0';i++){
    if(str[i]==ch){
        freq++;
    }
}
printf("frequency of the character '%c' in the string \"%s\" is %d",ch,str,freq);}
```

Task

- Write a program in C to convert a string to lowercase.

```
int main(){
char str[50];

gets(str);
for(int i=0 ;str[i]!='\0';i++){
    if(str[i]>='A'&&str[i]<='Z'){
        str[i]=str[i]+32;
    }
}
printf("the string after making lowercase \"%s\" ",str);

}
```

Task

- **Write a program in C to convert a string to uppercase.**

```
int main(){
char str[50];

gets(str);
for(int i=0 ;str[i]!='\0';i++){
    if(str[i]>='a'&&str[i]<='z'){
        str[i]=str[i]-32;
    }
}
printf("the string after making uppercase \"%s\" ",str);

}
```

Task

- **Take input of two strings and compare if they are same string or not**

```
int main()
{
    char str[100],str2[100];
    int len1,len2,i,flag=0;
    gets(str);
    gets(str2);
    len1=strlen(str);
    len2=strlen(str2);
```

```
    if(len1==len2){
        for(int i=0;i<len1;i++){
            if(str[i]!=str2[i]){
                flag=1;
                break;
            }
        }
        if(flag==1){
            printf("The two strings are not same");
        }
        else{
            printf("The two strings are same");
        }
    }
    else{
        printf("The two strings are not same");
    }
}
```

Task

- **Take a string as input and print it in reverse order.**

```
int main(){
    char str[100],str2[100];
    int len,i,j=0;
    gets(str); len=strlen(str);
    for(i=len-1;i>=0;i--)
    {
        str2[j++]=str[i];
    }
    str2[j]='\0'; puts(str2);
}
```

Task

- Take a string as input and check whether it is palindrome or not.

```
int main(){
    char str[100],str2[100];
    int len1,len2,i,flag=0,j=0;
    gets(str);
    len1=strlen(str);
    for(i=len1-1;i>=0;i--) {
        str2[j++]=str[i];
    }
    str2[j]='\0';

    len1=strlen(str);
    len2=strlen(str2);
```

```
    if(len1==len2){
        for(int i=0;i<len1;i++){
            if(str[i]!=str2[i]){
                flag=1;
                break;
            }
        }
        if(flag==1){
            printf("The two strings are not palindrome");
        }
        else{
            printf("The two strings are palindrome");
        }
    }
    else{
        printf("The two strings are not palindrome");
    }
}
```

Task

Take input of a string and count the number of unique characters

```
#include <stdio.h>
```

```
#include<string.h>
```

```
int main(){
```

```
char str1[50],uniq[50];
```

```
int flag=0,j;
```

```
gets(str1);
```

```
uniq[0]='\0';
```

```
for(int i=0;str1[i]!='\0';i++){  
    flag=0;  
    j=0;  
    for(;uniq[j]!='\0';j++){  
        if(str1[i]==uniq[j]){  
            flag=1;  
            break;  
        }  
    }  
    if(flag==0){ uniq[j] = str1[i];  
        uniq[j+1]='\0';  
    }  
}  
printf("unique characters %s",uniq);  
}
```

*Thank
you!*